

Custom Hooks: Fundamentos

Objetivos

- Entender qué son los custom hooks y por qué son útiles
- Aprender la anatomía de un custom hook
- Conocer las reglas para crear hooks
- Crear hooks básicos reutilizables

1. ¿Qué es un Custom Hook?

Anatomía de un Custom Hook

Un **custom hook** es una función de JavaScript/TypeScript que:

1. Comienza con el prefijo `use`
2. Puede llamar a otros hooks (`useState`, `useEffect`, etc.)
3. Encapsula lógica reutilizable entre componentes
4. Retorna valores y/o funciones

```
// =====
// QUÉ: Custom hook básico para toggle
// PARA: Reutilizar lógica de encendido/apagado
// IMPACTO: Evita duplicar código en componentes
// =====

import { useState, useCallback } from 'react';

// El nombre SIEMPRE empieza con "use"
const useToggle = (initialValue: boolean = false) => {
  // Puede usar otros hooks
  const [value, setValue] = useState(initialValue);

  // Funciones derivadas del estado
  const toggle = useCallback(() => setValue((prev) => !prev), []);
  const setTrue = useCallback(() => setValue(true), []);
  const setFalse = useCallback(() => setValue(false), []);

  // Retorna estado y funciones
  return { value, toggle, setTrue, setFalse };
};

export { useToggle };
```

Uso del Hook

```
// En cualquier componente
const Modal: React.FC = () => {
  const { value: isOpen, toggle, setFalse } = useToggle(false);

  return (
    <>
      <button onClick={toggle}>Abrir Modal</button>
    </>
  );
};
```

```

    {isOpen && (
      <div className="modal">
        <p>Contenido del modal</p>
        <button onClick={setFalse}>Cerrar</button>
      </div>
    )}
  </>
);
};

```

2. ¿Por Qué Custom Hooks?

✗ Sin Custom Hooks: Lógica Duplicada

```

// Componente A
const ComponentA: React.FC = () => {
  const [count, setCount] = useState(0);
  const increment = () => setCount((prev) => prev + 1);
  const decrement = () => setCount((prev) => prev - 1);
  const reset = () => setCount(0);
  // ...mismo código en otros componentes
};

// Componente B - DUPLICADO
const ComponentB: React.FC = () => {
  const [count, setCount] = useState(0);
  const increment = () => setCount((prev) => prev + 1);
  const decrement = () => setCount((prev) => prev - 1);
  const reset = () => setCount(0);
  // ...código idéntico
};

```

✓ Con Custom Hook: Lógica Reutilizable

```

// =====
// QUÉ: Custom hook para contador con opciones
// PARA: Reutilizar lógica de contador en múltiples componentes
// IMPACTO: Una sola implementación testeada para toda la app
// =====

// hooks/useCounter.ts
import { useState, useCallback } from 'react';

interface UseCounterOptions {
  min?: number;
  max?: number;
  step?: number;
}

interface UseCounterReturn {
  count: number;
  increment: () => void;
  decrement: () => void;
}

```

```

    reset: () => void;
    setCount: (value: number) => void;
  }

  const useCounter = (
    initialValue: number = 0,
    options: UseCounterOptions = {},
  ): UseCounterReturn => {
    const { min = -Infinity, max = Infinity, step = 1 } = options;
    const [count, setCountState] = useState(initialValue);

    const setCount = useCallback(
      (value: number) => {
        setCountState(Math.min(Math.max(value, min), max));
      },
      [min, max],
    );

    const increment = useCallback(() => {
      setCountState((prev) => Math.min(prev + step, max));
    }, [step, max]);

    const decrement = useCallback(() => {
      setCountState((prev) => Math.max(prev - step, min));
    }, [step, min]);

    const reset = useCallback(() => {
      setCountState(initialValue);
    }, [initialValue]);

    return { count, increment, decrement, reset, setCount };
  };

  export { useCounter };
  export type { UseCounterOptions, UseCounterReturn };

```

```

// Ahora en cualquier componente
const ComponentA: React.FC = () => {
  const { count, increment, decrement, reset } = useCounter(0, {
    min: 0,
    max: 10,
  });
  // ¡Lógica encapsulada y reutilizable!
};

const ComponentB: React.FC = () => {
  const { count, increment } = useCounter(5, { step: 5 });
  // Mismo hook, diferentes opciones
};

```

3. Anatomía de un Custom Hook

```
// =====
// ANATOMÍA COMPLETA DE UN CUSTOM HOOK
// =====

import { useState, useEffect, useCallback, useMemo } from 'react';

// 1 TIPOS: Definir interfaces para opciones y retorno
interface UseExampleOptions {
  initialValue?: string;
  validate?: (value: string) => boolean;
}

interface UseExampleReturn {
  value: string;
  setValue: (newValue: string) => void;
  isValid: boolean;
  reset: () => void;
}

// 2 HOOK: Función que empieza con "use"
const useExample = (options: UseExampleOptions = {}): UseExampleReturn => {
  // 3 DESTRUCTURING: Extraer opciones con defaults
  const { initialValue = '', validate = () => true } = options;

  // 4 ESTADO: Usar hooks nativos
  const [value, setValueState] = useState(initialValue);

  // 5 VALORES DERIVADOS: useMemo para cálculos
  const isValid = useMemo(() => validate(value), [value, validate]);

  // 6 CALLBACKS: useCallback para funciones estables
  const setValue = useCallback((newValue: string) => {
    setValueState(newValue);
  }, []);

  const reset = useCallback(() => {
    setValueState(initialValue);
  }, [initialValue]);

  // 7 EFECTOS: useEffect si es necesario
  useEffect(() => {
    // Efecto opcional
    console.log('Valor cambió:', value);
  }, [value]);

  // 8 RETORNO: Objeto con estado y funciones
  return { value, setValue, isValid, reset };
};

// 9 EXPORT: Nombrado para tree-shaking
export { useExample };
export type { UseExampleOptions, UseExampleReturn };
```

4. Reglas de los Hooks

Regla 1: Solo llamar hooks en el nivel superior

```
// ❌ MAL: Hook dentro de condicional
const useBadHook = (condition: boolean) => {
  if (condition) {
    const [value, setValue] = useState(''); // ERROR
  }
  return null;
};

// ✅ BIEN: Hook siempre se ejecuta
const useGoodHook = (condition: boolean) => {
  const [value, setValue] = useState('');

  // La condición va en la lógica, no en el hook
  const result = condition ? value : null;
  return result;
};
```

Regla 2: Solo llamar hooks desde funciones React

```
// ❌ MAL: Hook en función regular
function regularFunction() {
  const [value] = useState(''); // ERROR
}

// ✅ BIEN: Hook en componente o custom hook
const MyComponent: React.FC = () => {
  const [value] = useState(''); // OK en componente
  return <div>{value}</div>;
};

const useMyHook = () => {
  const [value] = useState(''); // OK en custom hook
  return value;
};
```

Regla 3: El nombre debe empezar con "use"

```
// ❌ MAL: No empieza con "use"
const getCounter = () => {
  const [count, setCount] = useState(0);
  return { count, setCount };
};

// ✅ BIEN: Empieza con "use"
const useCounter = () => {
  const [count, setCount] = useState(0);
};
```

```
    return { count, setCount };
  };
}
```

5. Tipado de Custom Hooks

Tipos Explícitos para Retorno

```
// =====
// QUÉ: Hook useInput con tipado explícito
// PARA: Manejar inputs de forma reutilizable y type-safe
// IMPACTO: Autocompletado en IDE y errores en tiempo de compilación
// =====

// Definir interface para el retorno
interface UseInputReturn {
  value: string;
  onChange: (e: React.ChangeEvent<HTMLInputElement>) => void;
  reset: () => void;
  bind: {
    value: string;
    onChange: (e: React.ChangeEvent<HTMLInputElement>) => void;
  };
}

// Especificar tipo de retorno
const useInput = (initialValue: string = ''): UseInputReturn => {
  const [value, setValue] = useState(initialValue);

  const onChange = useCallback((e: React.ChangeEvent<HTMLInputElement>) => {
    setValue(e.target.value);
  }, []);

  const reset = useCallback(() => {
    setValue(initialValue);
  }, [initialValue]);

  // Objeto para spread en input: <input {...bind} />
  const bind = useMemo(() => ({ value, onChange }), [value, onChange]);

  return { value, onChange, reset, bind };
};
```

Uso con TypeScript

```
const LoginForm: React.FC = () => {
  const email = useInput('');
  const password = useInput('');

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    console.log('Login:', email.value, password.value);
    email.reset();
    password.reset();
  };
}
```

```

});

return (
  <form onSubmit={handleSubmit}>
    {/* Spread del objeto bind */}
    <input
      type="email"
      placeholder="Email"
      {...email.bind}
    />
    <input
      type="password"
      placeholder="Password"
      {...password.bind}
    />
    <button type="submit">Login</button>
  </form>
);
};

```

6. Hook vs Función Regular

Característica	Custom Hook	Función Regular
Puede usar hooks	✅ Sí	❌ No
Nombre	useSomething	getSomething
Estado reactivo	✅ Causa re-renders	❌ No reactivo
Efectos secundarios	✅ Con useEffect	❌ Sin ciclo de vida
Compartir estado	Cada instancia es única	Sin estado

```

// =====
// QUÉ: Comparación entre hook y función regular
// PARA: Entender cuándo usar cada uno
// IMPACTO: Elegir la herramienta correcta para cada caso
// =====

// FUNCIÓN REGULAR: Solo calcula, no tiene estado
const formatDate = (date: Date): string => {
  return date.toLocaleDateString('es-ES');
};

// CUSTOM HOOK: Tiene estado y causa re-renders
const useCurrentDate = () => {
  const [date, setDate] = useState(new Date());

  useEffect(() => {
    const timer = setInterval(() => {
      setDate(new Date());
    }, 1000);

    return () => clearInterval(timer);
  }, []);
};

```

```

    return date;
  };

  // USO
  const DateDisplay: React.FC = () => {
    const currentDate = useCurrentDate(); // Re-render cada segundo
    const formatted = formatDate(currentDate); // Solo formatea

    return <p>Fecha: {formatted}</p>;
  };

```

7. Ejemplos Básicos Completos

useBoolean

```

// =====
// QUÉ: Hook useBoolean para estados booleanos
// PARA: Simplificar toggles, modales, dropdowns, etc.
// IMPACTO: API clara y consistente para estados on/off
// =====

import { useState, useCallback } from 'react';

interface UseBooleanReturn {
  value: boolean;
  setTrue: () => void;
  setFalse: () => void;
  toggle: () => void;
  setValue: (value: boolean) => void;
}

const useBoolean = (initialValue: boolean = false): UseBooleanReturn => {
  const [value, setValue] = useState(initialValue);

  const setTrue = useCallback(() => setValue(true), []);
  const setFalse = useCallback(() => setValue(false), []);
  const toggle = useCallback(() => setValue((prev) => !prev), []);

  return { value, setTrue, setFalse, toggle, setValue };
};

export { useBoolean };

```

usePrevious

```

// =====
// QUÉ: Hook usePrevious para recordar valor anterior
// PARA: Comparar estados entre renders
// IMPACTO: Útil para animaciones, validaciones, analytics
// =====

import { useRef, useEffect } from 'react';

```



```

const usePrevious = <T,>(value: T): T | undefined => {
  // useRef persiste entre renders sin causar re-render
  const ref = useRef<T | undefined>(undefined);

  useEffect(() => {
    ref.current = value;
  }, [value]);

  // Retorna el valor ANTERIOR (antes de este render)
  return ref.current;
};

export { usePrevious };

// Uso
const Counter: React.FC = () => {
  const [count, setCount] = useState(0);
  const previousCount = usePrevious(count);

  return (
    <div>
      <p>Actual: {count}</p>
      <p>Anterior: {previousCount ?? 'N/A'}</p>
      <button onClick={() => setCount(c => c + 1)}>+1</button>
    </div>
  );
};

```

useUpdateEffect

```

// =====
// QUÉ: Hook useUpdateEffect que ignora el primer render
// PARA: Ejecutar efectos solo en actualizaciones
// IMPACTO: Evita lógica innecesaria en el mount inicial
// =====

import { useEffect, useRef } from 'react';

// useEffect que NO se ejecuta en el primer render
const useUpdateEffect = (
  effect: React.EffectCallback,
  deps?: React.DependencyList
): void => {
  const isFirstRender = useRef(true);

  useEffect(() => {
    if (isFirstRender.current) {
      isFirstRender.current = false;
      return;
    }

    return effect();
  }, deps);

```

```
};

export { useUpdateEffect };

// Uso: Solo ejecuta cuando cambia, no en mount
const SearchResults: React.FC<{ query: string }> = ({ query }) => {
  useUpdateEffect(() => {
    console.log('Query cambió a:', query);
    // Fetch nuevos resultados...
  }, [query]);

  return <div>Resultados para: {query}</div>;
};
```

Recursos Adicionales

- [React Docs: Custom Hooks](#)
- [useHooks.com](#)
- [React TypeScript Cheatsheet: Hooks](#)

Checklist de Comprensión

- ☐ Entiendo qué es un custom hook y para qué sirve
- ☐ Conozco las 3 reglas de los hooks
- ☐ Sé la diferencia entre hook y función regular
- ☐ Puedo tipar correctamente un custom hook
- ☐ Entiendo cuándo usar useCallback en hooks

Siguiente: [02:patrones-custom-hooks.md](#)

Patrones de Custom Hooks

Objetivos

- Aprender patrones comunes para hooks reutilizables
- Implementar hooks con estado complejo
- Crear hooks con efectos y cleanup
- Entender composición de hooks

 Composición de Hooks

1. Patrón: Hook de Estado con Acciones

Hooks que encapsulan estado y proveen acciones específicas.

```
// =====
// QUÉ: Hook useArray para manejar arrays con operaciones CRUD
// PARA: Gestionar listas (todos, carrito, favoritos, etc.)
// IMPACTO: API limpia que encapsula mutaciones inmutables
// =====
```

```

import { useState, useCallback } from 'react';

interface UseArrayReturn<T> {
  array: T[];
  set: (newArray: T[]) => void;
  push: (element: T) => void;
  filter: (callback: (element: T) => boolean) => void;
  update: (index: number, newElement: T) => void;
  remove: (index: number) => void;
  clear: () => void;
}

const useArray = <T> (initialValue: T[] = []): UseArrayReturn<T> => {
  const [array, setArray] = useState<T[]>(initialValue);

  const push = useCallback((element: T) => {
    setArray((prev) => [...prev, element]);
  }, []);

  const filter = useCallback((callback: (element: T) => boolean) => {
    setArray((prev) => prev.filter(callback));
  }, []);

  const update = useCallback((index: number, newElement: T) => {
    setArray((prev) => {
      const copy = [...prev];
      copy[index] = newElement;
      return copy;
    });
  }, []);

  const remove = useCallback((index: number) => {
    setArray((prev) => prev.filter((_, i) => i !== index));
  }, []);

  const clear = useCallback(() => {
    setArray([]);
  }, []);

  return { array, set: setArray, push, filter, update, remove, clear };
};

export { useArray };
export type { UseArrayReturn };

```

Uso del Patrón

```

interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

const Todolist: React.FC = () => {

```

```

const todos = useArray<Todo>([]);
const [inputValue, setInputValue] = useState('');

const addTodo = () => {
  if (!inputValue.trim()) return;
  todos.push({
    id: Date.now(),
    text: inputValue,
    completed: false,
  });
  setInputValue('');
};

const toggleTodo = (index: number) => {
  const todo = todos.array[index];
  todos.update(index, { ...todo, completed: !todo.completed });
};

return (
  <div>
    <input
      value={inputValue}
      onChange={(e) => setInputValue(e.target.value)}
      onKeyDown={(e) => e.key === 'Enter' && addTodo()}
    />
    <button onClick={addTodo}>Agregar</button>

    <ul>
      {todos.array.map((todo, index) => (
        <li key={todo.id}>
          <span
            style={{
              textDecoration: todo.completed ? 'line-through' : 'none',
            }}
            onClick={() => toggleTodo(index)}>
            {todo.text}
          </span>
          <button onClick={() => todos.remove(index)}>✖</button>
        </li>
      ))}
    </ul>

    <button onClick={() => todos.filter((t) => !t.completed)}>
      Limpiar completados
    </button>
    <button onClick={todos.clear}>Limpiar todo</button>
  </div>
);
};

```

2. Patrón: Hook con Efectos

Hooks que encapsulan efectos secundarios con cleanup.

```
// =====
// QUÉ: Hook useDebounce para retrasar actualizaciones
// PARA: Optimizar búsquedas, filtros y llamadas a API
// IMPACTO: Reduce llamadas innecesarias mientras el usuario escribe
// =====

import { useState, useEffect } from 'react';

const useDebounce = <T>(value: T, delay: number = 500): T => {
  const [debouncedValue, setDebouncedValue] = useState<T>(value);

  useEffect(() => {
    // Crear timer que actualiza después del delay
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    // Cleanup: cancelar timer si value cambia antes del delay
    return () => {
      clearTimeout(timer);
    };
  }, [value, delay]);

  return debouncedValue;
};

export { useDebounce };
```

Uso para Búsqueda

```
const SearchComponent: React.FC = () => {
  const [searchTerm, setSearchTerm] = useState('');
  const debouncedSearchTerm = useDebounce(searchTerm, 300);

  // Este efecto solo se ejecuta 300ms después de dejar de escribir
  useEffect(() => {
    if (debouncedSearchTerm) {
      console.log('Buscando:', debouncedSearchTerm);
      // fetch(`/api/search?q=${debouncedSearchTerm}`) ...
    }
  }, [debouncedSearchTerm]);

  return (
    <input
      value={searchTerm}
      onChange={(e) => setSearchTerm(e.target.value)}
      placeholder="Buscar..."
    />
  );
};
```

3. Patrón: Hook de Fetch con Estados

Hook completo para fetching con loading, error y data.

```
// =====
// QUÉ: Hook useFetch para llamadas HTTP
// PARA: Manejar estados de carga, error y datos de forma consistente
// IMPACTO: Elimina boilerplate repetitivo en cada componente
// =====

import { useState, useEffect, useCallback } from 'react';

interface UseFetchState<T> {
  data: T | null;
  loading: boolean;
  error: Error | null;
}

interface UseFetchReturn<T> extends UseFetchState<T> {
  refetch: () => Promise<void>;
}

const useFetch = <T>(url: string): UseFetchReturn<T> => {
  const [state, setState] = useState<UseFetchState<T>>({
    data: null,
    loading: true,
    error: null,
  });

  const fetchData = useCallback(async () => {
    setState((prev) => ({ ...prev, loading: true, error: null }));

    try {
      const response = await fetch(url);

      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }

      const data = await response.json();
      setState({ data, loading: false, error: null });
    } catch (error) {
      setState({
        data: null,
        loading: false,
        error: error instanceof Error ? error : new Error('Error desconocido'),
      });
    }
  }, [url]);

  useEffect(() => {
    fetchData();
  }, [fetchData]);

  return { ...state, refetch: fetchData };
};
```

```
export { useFetch };
export type { UseFetchReturn };
```

Versión Mejorada con Cancelación

```
import { useState, useEffect, useCallback, useRef } from 'react';

interface UseFetchOptions {
  enabled?: boolean; // Permite deshabilitar el fetch
}

const useFetchAdvanced = <T>(  
  url: string,  
  options: UseFetchOptions = {},  
) : UseFetchReturn<T> => {  
  const { enabled = true } = options;  
  const abortControllerRef = useRef<AbortController | null>(null);  
  
  const [state, setState] = useState<UseFetchState<T>>({  
    data: null,  
    loading: enabled,  
    error: null,  
  }));  
  
  const fetchData = useCallback(async () => {  
    // Cancelar request anterior si existe  
    if (abortControllerRef.current) {  
      abortControllerRef.current.abort();  
    }  
  
    // Crear nuevo AbortController  
    abortControllerRef.current = new AbortController();  
    const { signal } = abortControllerRef.current;  
  
    setState((prev) => ({ ...prev, loading: true, error: null }));  
  
    try {  
      const response = await fetch(url, { signal });  
  
      if (!response.ok) {  
        throw new Error(`HTTP error! status: ${response.status}`);  
      }  
  
      const data = await response.json();  
      setState({ data, loading: false, error: null });  
    } catch (error) {  
      // Ignorar errores de cancelación  
      if (error instanceof Error && error.name === 'AbortError') {  
        return;  
      }  
      setState({  
        data: null,  
        loading: false,  
        error: error instanceof Error ? error : new Error('Error desconocido'),  
      });  
    }  
  }, [url, options]);  
  
  if (enabled) {  
    fetchData();  
  }  
};
```

```

    }
  }, [url]);

  useEffect(() => {
    if (enabled) {
      fetchData();
    }

    // Cleanup: cancelar request si el componente se desmonta
    return () => {
      if (abortControllerRef.current) {
        abortControllerRef.current.abort();
      }
    };
  }, [fetchData, enabled]);

  return { ...state, refetch: fetchData };
};

export { useFetchAdvanced };

```

Uso del Hook Fetch

```

interface User {
  id: number;
  name: string;
  email: string;
}

const UserProfile: React.FC<{ userId: number }> = ({ userId }) => {
  const {
    data: user,
    loading,
    error,
    refetch,
  } = useFetch<User>(`https://jsonplaceholder.typicode.com/users/${userId}`);

  if (loading) return <p>Cargando...</p>;
  if (error) return <p>Error: {error.message}</p>;
  if (!user) return <p>No se encontró usuario</p>;

  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.email}</p>
      <button onClick={refetch}>Actualizar</button>
    </div>
  );
};

```

4. Patrón: Hook de Formulario

Hook para manejar formularios con validación.


```
// =====
// QUÉ: Hook useForm para formularios con validación
// PARA: Manejar estado, errores, touched y submit de forms
// IMPACTO: Formularios robustos sin dependencias externas
// =====

import { useState, useCallback, ChangeEvent, FormEvent } from 'react';

type ValidationRule<T> = (value: T) => string | null;

interface UseFormOptions<T extends Record<string, unknown>> {
  initialValues: T;
  validations?: Partial<Record<keyof T, ValidationRule<T[keyof T]>>>;
  onSubmit: (values: T) => void | Promise<void>;
}

interface UseFormReturn<T extends Record<string, unknown>> {
  values: T;
  errors: Partial<Record<keyof T, string>>;
  touched: Partial<Record<keyof T, boolean>>;
  isSubmitting: boolean;
  isValid: boolean;
  handleChange: (
    e: ChangeEvent<HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement>,
  ) => void;
  handleBlur: (
    e: ChangeEvent<HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement>,
  ) => void;
  handleSubmit: (e: FormEvent) => Promise<void>;
  setFieldValue: (field: keyof T, value: T[keyof T]) => void;
  reset: () => void;
}

const useForm = <T extends Record<string, unknown>>(<
  options: UseFormOptions<T>,
>): UseFormReturn<T> => {
  const { initialValues, validations = {}, onSubmit } = options;

  const [values, setValues] = useState<T>(initialValues);
  const [errors, setErrors] = useState<Partial<Record<keyof T, string>>>({});
  const [touched, setTouched] = useState<Partial<Record<keyof T, boolean>>>({});
  const [isSubmitting, setIsSubmitting] = useState(false);

  // Validar un campo específico
  const validateField = useCallback(
    (field: keyof T, value: T[keyof T]): string | null => {
      const validation = validations[field];
      if (validation) {
        return validation(value);
      }
      return null;
    },
    [validations],
  );

  // Validar todos los campos
```

```

const validateAll = useCallback(() : boolean => {
  const newErrors: Partial<Record<keyof T, string>> = {};
  let isValid = true;

  for (const field of Object.keys(values) as Array<keyof T>) {
    const error = validateField(field, values[field]);
    if (error) {
      newErrors[field] = error;
      isValid = false;
    }
  }

  setErrors(newErrors);
  return isValid;
}, [values, validateField]);

const handleChange = useCallback(
  (
    e: ChangeEvent<
      HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement
    >,
  ) => {
    const { name, value, type } = e.target;
    const fieldValue =
      type === 'checkbox' ? (e.target as HTMLInputElement).checked : value;

    setValues((prev) => ({ ...prev, [name]: fieldValue }));

    // Validar mientras escribe si ya tocó el campo
    if (touched[name as keyof T]) {
      const error = validateField(name as keyof T, fieldValue as T[keyof T]);
      setErrors((prev) => ({ ...prev, [name]: error ?? undefined }));
    }
  },
  [touched, validateField],
);

const handleBlur = useCallback(
  (
    e: ChangeEvent<
      HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement
    >,
  ) => {
    const { name, value } = e.target;
    setTouched((prev) => ({ ...prev, [name]: true }));

    const error = validateField(name as keyof T, value as T[keyof T]);
    setErrors((prev) => ({ ...prev, [name]: error ?? undefined }));
  },
  [validateField],
);

const handleSubmit = useCallback(
  async (e: FormEvent) => {
    e.preventDefault();
  },
);

```

```

// Marcar todos como tocados
const allTouched = Object.keys(values).reduce(
  (acc, key) => {
    acc[key as keyof T] = true;
    return acc;
  },
  {} as Record<keyof T, boolean>,
);
setTouched(allTouched);

// Validar todo
if (!validateAll()) {
  return;
}

setIsSubmitting(true);
try {
  await onSubmit(values);
} finally {
  setIsSubmitting(false);
}
},
[values, validateAll, onSubmit],
);

const setFieldValue = useCallback((field: keyof T, value: T[keyof T]) => {
  setValues((prev) => ({ ...prev, [field]: value }));
}, []);

const reset = useCallback(() => {
  setValues(initialValues);
  setErrors({});
  setTouched({});
  setIsSubmitting(false);
}, [initialValues]);

const isValid = Object.values(errors).every((error) => !error);

return {
  values,
  errors,
  touched,
  isSubmitting,
  isValid,
  handleChange,
  handleBlur,
  handleSubmit,
  setFieldValue,
  reset,
};
};

export { useForm };
export type { UseFormReturn, UseFormOptions, ValidationRule };

```

Uso del Hook Form

```
interface LoginForm {
  email: string;
  password: string;
  remember: boolean;
}

const LoginPage: React.FC = () => {
  const form = useForm<LoginForm>({
    initialValues: {
      email: '',
      password: '',
      remember: false,
    },
    validations: {
      email: (value) => {
        if (!value) return 'El email es requerido';
        if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value as string)) {
          return 'Email inválido';
        }
        return null;
      },
      password: (value) => {
        if (!value) return 'La contraseña es requerida';
        if ((value as string).length < 6) {
          return 'Mínimo 6 caracteres';
        }
        return null;
      },
    },
    onSubmit: async (values) => {
      console.log('Login:', values);
      // await loginUser(values);
    },
  });

  return (
    <form onSubmit={form.handleSubmit}>
      <div>
        <input
          name="email"
          type="email"
          value={form.values.email}
          onChange={form.handleChange}
          onBlur={form.handleBlur}
          placeholder="Email"
        />
        {form.touched.email && form.errors.email && (
          <span className="error">{form.errors.email}</span>
        )}
      </div>

      <div>
        <input
```

```

        name="password"
        type="password"
        value={form.values.password}
        onChange={form.handleChange}
        onBlur={form.handleBlur}
        placeholder="Contraseña"
      />
      {form.touched.password && form.errors.password && (
        <span className="error">{form.errors.password}</span>
      )}
    </div>

    <label>
      <input
        name="remember"
        type="checkbox"
        checked={form.values.remember}
        onChange={form.handleChange}
      />
      Recordarme
    </label>

    <button
      type="submit"
      disabled={form.isSubmitting}>
      {form.isSubmitting ? 'Cargando...' : 'Iniciar sesión'}
    </button>
  </form>
);
};

```

5. Patrón: Composición de Hooks

Hooks que usan otros hooks internamente.

```

// =====
// QUÉ: Hook useAuth que compone useLocalStorage + estado
// PARA: Manejar autenticación con persistencia automática
// IMPACTO: Lógica de auth encapsulada y reutilizable
// =====

import { useMemo } from 'react';

// Hooks base
const useLocalStorage = <T>(key: string, initialValue: T) => {
  const [storedValue, setStoredValue] = useState<T>(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch {
      return initialValue;
    }
  });

  const setValue = useCallback(

```

```

    (value: T | ((prev: T) => T)) => {
      const valueToStore =
        value instanceof Function ? value(storedValue) : value;
      setStoredValue(valueToStore);
      window.localStorage.setItem(key, JSON.stringify(valueToStore));
    },
    [key, storedValue],
  );

  return [storedValue, setValue] as const;
};

// Hook compuesto que usa useLocalStorage + useFetch
interface User {
  id: number;
  name: string;
  email: string;
}

interface AuthState {
  user: User | null;
  token: string | null;
}

interface UseAuthReturn {
  user: User | null;
  token: string | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
}

const useAuth = (): UseAuthReturn => {
  // Usar hook de localStorage para persistencia
  const [authState, setAuthState] = useLocalStorage<AuthState>('auth', {
    user: null,
    token: null,
  });

  const [isLoading, setIsLoading] = useState(false);

  const isAuthenticated = useMemo(() => {
    return authState.token !== null && authState.user !== null;
  }, [authState]);

  const login = useCallback(
    async (email: string, password: string) => {
      setIsLoading(true);
      try {
        // Simular llamada a API
        const response = await fetch('/api/login', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ email, password }),
        });
      }
    }
  );

```

```

    if (!response.ok) {
      throw new Error('Credenciales inválidas');
    }

    const { user, token } = await response.json();
    setAuthState({ user, token });
  } finally {
    setIsLoading(false);
  }
},
[setAuthState],
);

const logout = useCallback(() => {
  setAuthState({ user: null, token: null });
  // Opcional: llamar endpoint de logout
}, [setAuthState]);

return {
  user: authState.user,
  token: authState.token,
  isAuthenticated,
  isLoading,
  login,
  logout,
};
};

export { useAuth };
export type { UseAuthReturn };

```

6. Patrón: Hook con Reducer

Para estado complejo con múltiples transiciones.

```

// =====
// QUÉ: Hook useCart con useReducer para estado complejo
// PARA: Manejar carrito de compras con múltiples acciones
// IMPACTO: Estado predecible, fácil de testear y debuggear
// =====

import { useReducer, useCallback, useMemo } from 'react';

// Estado del carrito
interface CartItem {
  id: string;
  name: string;
  price: number;
  quantity: number;
}

interface CartState {
  items: CartItem[];
  isOpen: boolean;
}

```

```

}

// Acciones discriminadas
type CartAction =
  | { type: 'ADD_ITEM'; payload: Omit<CartItem, 'quantity'> }
  | { type: 'REMOVE_ITEM'; payload: string }
  | { type: 'UPDATE_QUANTITY'; payload: { id: string; quantity: number } }
  | { type: 'CLEAR_CART' }
  | { type: 'TOGGLE_CART' };

// Reducer puro
const cartReducer = (state: CartState, action: CartAction): CartState => {
  switch (action.type) {
    case 'ADD_ITEM': {
      const existingItem = state.items.find(
        (item) => item.id === action.payload.id,
      );
      if (existingItem) {
        return {
          ...state,
          items: state.items.map((item) =>
            item.id === action.payload.id
              ? { ...item, quantity: item.quantity + 1 }
              : item,
          ),
        };
      }
      return {
        ...state,
        items: [...state.items, { ...action.payload, quantity: 1 }],
      };
    }
    case 'REMOVE_ITEM':
      return {
        ...state,
        items: state.items.filter((item) => item.id !== action.payload),
      };
    case 'UPDATE_QUANTITY':
      return {
        ...state,
        items: state.items.map((item) =>
          item.id === action.payload.id
            ? { ...item, quantity: action.payload.quantity }
            : item,
        ),
      };
    case 'CLEAR_CART':
      return { ...state, items: [] };
    case 'TOGGLE_CART':
      return { ...state, isOpen: !state.isOpen };
    default:
      return state;
  }
};

// Hook que encapsula el reducer

```



```

interface UseCartReturn {
  items: CartItem[];
  isOpen: boolean;
  total: number;
  itemCount: number;
  addItem: (item: Omit<CartItem, 'quantity'>) => void;
  removeItem: (id: string) => void;
  updateQuantity: (id: string, quantity: number) => void;
  clearCart: () => void;
  toggleCart: () => void;
}

const useCart = (): UseCartReturn => {
  const [state, dispatch] = useReducer(cartReducer, {
    items: [],
    isOpen: false,
  });

  // Valores calculados con useMemo
  const total = useMemo(() => {
    return state.items.reduce(
      (sum, item) => sum + item.price * item.quantity,
      0,
    );
  }, [state.items]);

  const itemCount = useMemo(() => {
    return state.items.reduce((sum, item) => sum + item.quantity, 0);
  }, [state.items]);

  // Action creators envueltos en useCallback
  const addItem = useCallback((item: Omit<CartItem, 'quantity'>) => {
    dispatch({ type: 'ADD_ITEM', payload: item });
  }, []);

  const removeItem = useCallback((id: string) => {
    dispatch({ type: 'REMOVE_ITEM', payload: id });
  }, []);

  const updateQuantity = useCallback((id: string, quantity: number) => {
    dispatch({ type: 'UPDATE_QUANTITY', payload: { id, quantity } });
  }, []);

  const clearCart = useCallback(() => {
    dispatch({ type: 'CLEAR_CART' });
  }, []);

  const toggleCart = useCallback(() => {
    dispatch({ type: 'TOGGLE_CART' });
  }, []);

  return {
    items: state.items,
    isOpen: state.isOpen,
    total,
    itemCount,
  };
}

```

```
    addItem,  
    removeItem,  
    updateQuantity,  
    clearCart,  
    toggleCart,  
  };  
};  
  
export { useCart };  
export type { UseCartReturn, CartItem };
```

Recursos Adicionales

- [Patterns.dev: Hooks Patterns](#)
- [Kent C. Dodds: Advanced React Patterns](#)
- [useHooks.com](#)

Checklist de Comprensión

- ☐ Conozco el patrón Estado + Acciones
- ☐ Sé crear hooks con efectos y cleanup
- ☐ Puedo implementar un hook de fetch completo
- ☐ Entiendo la composición de hooks
- ☐ Sé cuándo usar useReducer en un hook

Siguiente: [03-ciclo-vida-componentes.md](#)

Ciclo de Vida de Componentes con Hooks

Objetivos

- Entender el ciclo de vida de componentes funcionales
- Mapear métodos de clases a hooks equivalentes
- Dominar useEffect para cada fase del ciclo de vida
- Evitar errores comunes con efectos

1. Ciclo de Vida en React

 Ciclo de Vida de Componentes

Componentes de Clase (Legado)

```
// ✗ FORMA ANTIGUA - Solo referencia  
class MyComponent extends React.Component {  
  componentDidMount() {  
    // Se ejecuta DESPUÉS del primer render  
  }  
  
  componentDidUpdate(prevProps, prevState) {  
    // Se ejecuta DESPUÉS de cada actualización  
  }  
}
```

```

}

componentWillUnmount() {
  // Se ejecuta ANTES de desmontar
}
}

```

Componentes Funcionales (Moderno)

```

// ✅ FORMA MODERNA - Con hooks
const MyComponent: React.FC = () => {
  // 1. MOUNT: Después del primer render
  useEffect(() => {
    console.log('Componente montado');
  }, []); // Array vacío = solo en mount

  // 2. UPDATE: Después de cada render
  useEffect(() => {
    console.log('Componente renderizado');
  }); // Sin dependencias = cada render

  // 3. UNMOUNT: Antes de desmontar
  useEffect(() => {
    return () => {
      console.log('Componente se desmontará');
    };
  }, []);

  return <div>Mi Componente</div>;
};

```

2. Fases del Ciclo de Vida

Fase 1: Montaje (Mount)

```

// =====
// QUÉ: useEffect con array vacío para fase de montaje
// PARA: Ejecutar código una sola vez al montar el componente
// IMPACTO: Equivalente a componentDidMount en clases
// =====

const MountExample: React.FC = () => {
  const [data, setData] = useState<string | null>(null);

  // componentDidMount equivalente
  useEffect(() => {
    console.log('✅ Componente montado');

    // Típico: fetch inicial de datos
    const fetchData = async () => {
      const response = await fetch('/api/data');
      const result = await response.json();
      setData(result);
    };
  }, []);
};

```

```

};

fetchData();

// Típico: suscripciones
const subscription = someService.subscribe();

// Cleanup para cuando se desmonte
return () => {
  subscription.unsubscribe();
};
}, []); // ← Array vacío = solo mount

return <div>{data}</div>;
};

```

| Tiempo de inicio | < 1 segundo | 10-30 segundos | | HMR | Instantáneo | 1-5 segundos | | Configuración | vite.config.ts | Eject necesario | | Build tool | Rollup (ESBuild) | Webpack | | Tamaño bundle | Más pequeño | Más grande |

Fase 2: Actualización (Update)

```

// =====
// QUÉ: useEffect con dependencias para fase de actualización
// PARA: Re-ejecutar lógica cuando cambian valores específicos
// IMPACTO: Control granular sobre cuándo se ejecutan efectos
// =====

interface UserProfileProps {
  userId: number;
}

const UserProfile: React.FC<UserProfileProps> = ({ userId }) => {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  // componentDidUpdate equivalente (para userId)
  useEffect(() => {
    console.log('🔄 userId cambió, re-fetching...');

    let isCancelled = false;

    const fetchUser = async () => {
      setLoading(true);
      try {
        const response = await fetch(`/api/users/${userId}`);
        const data = await response.json();

        // Verificar que no se canceló
        if (!isCancelled) {
          setUser(data);
        }
      } finally {
        if (!isCancelled) {
          setLoading(false);
        }
      }
    };
  }, [userId]);
};

```

```

    }
  };

  fetchUser();

  // Cleanup: marcar como cancelado si userId cambia antes
  return () => {
    isCancelled = true;
  };
}, [userId]); // ← Se ejecuta cuando userId cambia

if (loading) return <p>Cargando...</p>;
return <div>{user?.name}</div>;
};

```

Fase 3: Desmontaje (Unmount)

```

// =====
// QUÉ: Función de cleanup en useEffect para desmontaje
// PARA: Limpiar recursos (timers, suscripciones, listeners)
// IMPACTO: Previene memory leaks y comportamiento inesperado
// =====

const TimerComponent: React.FC = () => {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    console.log('🕒 Timer iniciado');

    const interval = setInterval(() => {
      setSeconds((prev) => prev + 1);
    }, 1000);

    // componentWillUnmount equivalente
    return () => {
      console.log('🛑 Timer detenido (cleanup)');
      clearInterval(interval);
    };
  }, []);

  return <p>Segundos: {seconds}</p>;
};

// Componente padre que monta/desmonta
const App: React.FC = () => {
  const [showTimer, setShowTimer] = useState(true);

  return (
    <div>
      <button onClick={() => setShowTimer((prev) => !prev)}>
        {showTimer ? 'Ocultar' : 'Mostrar'} Timer
      </button>

      {showTimer && <TimerComponent />}
      {/* Al ocultar, se ejecuta el cleanup del timer */}
    </div>
  );
};

```

```
    </div>
  );
};
```

3. Mapa de Equivalencias

Método de Clase	Hook Equivalente
constructor	useState (estado inicial)
componentDidMount	useEffect(() => {}, [])
componentDidUpdate	useEffect(() => {}, [deps])
componentWillUnmount	useEffect(() => { return () => {} }, [])
shouldComponentUpdate	React.memo, useMemo, useCallback
getDerivedStateFromProps	useState + lógica en render
getSnapshotBeforeUpdate	useLayoutEffect (casos especiales)

4. useEffect vs useLayoutEffect

```
// =====
// QUÉ: Diferencia entre useEffect y useLayoutEffect
// PARA: Elegir el hook correcto según el caso de uso
// IMPACTO: useLayoutEffect evita parpadeos visuales en DOM
// =====

import { useEffect, useLayoutEffect, useState, useRef } from 'react';

const LayoutEffectExample: React.FC = () => {
  const [value, setValue] = useState(0);
  const boxRef = useRef<HTMLDivElement>(null);

  // ❌ useEffect: Puede causar "flash" visual
  useEffect(() => {
    if (boxRef.current) {
      // Se ejecuta DESPUÉS de que el usuario ve el DOM
      boxRef.current.style.transform = `translateX(${value * 100}px)`;
    }
  }, [value]);

  // ✅ useLayoutEffect: Sin "flash", se ejecuta ANTES del paint
  useLayoutEffect(() => {
    if (boxRef.current) {
      // Se ejecuta ANTES de que el usuario vea cambios
      // Útil para medir DOM o hacer animaciones inmediatas
      const rect = boxRef.current.getBoundingClientRect();
      console.log('Posición:', rect.left);
    }
  }, [value]);

  return (
```

```

    <div>
      <div
        ref={boxRef}
        className="box">
          Box
        </div>
      <button onClick={() => setValue((v) => v + 1)}>Mover</button>
    </div>
  );
};

```

Cuándo usar cada uno

Caso de Uso	Hook Recomendado
Fetch de datos	useEffect
Suscripciones	useEffect
Timers	useEffect
Medir elementos DOM	useLayoutEffect
Sincronizar scroll	useLayoutEffect
Tooltips que dependen de posición	useLayoutEffect

5. Orden de Ejecución

```

// =====
// QUÉ: Demostración del orden de ejecución de efectos
// PARA: Entender cómo React procesa componentes padre/hijo
// IMPACTO: Debuggear problemas de timing en efectos
// =====

const Parent: React.FC = () => {
  console.log('1 Parent: render');

  useLayoutEffect(() => {
    console.log('4 Parent: useLayoutEffect');
    return () => console.log('Parent: cleanup layoutEffect');
  }, []);

  useEffect(() => {
    console.log('6 Parent: useEffect');
    return () => console.log('Parent: cleanup effect');
  }, []);

  return (
    <div>
      <Child />
    </div>
  );
};

const Child: React.FC = () => {

```

```

console.log('2 Child: render');

useLayoutEffect(() => {
  console.log('3 Child: useLayoutEffect');
  return () => console.log('Child: cleanup layoutEffect');
}, []);

useEffect(() => {
  console.log('5 Child: useEffect');
  return () => console.log('Child: cleanup effect');
}, []);

return <p>Child Component</p>;
};

// ORDEN DE CONSOLA:
// 1 Parent: render
// 2 Child: render
// 3 Child: useLayoutEffect (hijos primero)
// 4 Parent: useLayoutEffect
// 5 Child: useEffect (hijos primero)
// 6 Parent: useEffect

```

6. Patrones Comunes del Ciclo de Vida

Patrón: Efecto con Dependencias Múltiples

```

interface SearchParams {
  query: string;
  page: number;
  filters: string[];
}

const SearchResults: React.FC<SearchParams> = ({ query, page, filters }) => {
  const [results, setResults] = useState<Item[]>([]);
  const [loading, setLoading] = useState(false);

  // Se ejecuta cuando CUALQUIERA de las dependencias cambia
  useEffect(() => {
    const abortController = new AbortController();

    const search = async () => {
      setLoading(true);
      try {
        const params = new URLSearchParams({
          q: query,
          page: String(page),
          filters: filters.join(','),
        });

        const response = await fetch(`/api/search?${params}`, {
          signal: abortController.signal,
        });
      }
    };
  }, [query, page, filters]);

```



```

    const data = await response.json();
    setResults(data);
  } catch (error) {
    if (error instanceof Error && error.name !== 'AbortError') {
      console.error('Error en búsqueda:', error);
    }
  } finally {
    setLoading(false);
  }
};

search();

return () => {
  abortController.abort();
};
}, [query, page, filters]); // Todas las dependencias

return (
  <div>
    {loading ? (
      <p>Buscando...</p>
    ) : (
      <ul>
        {results.map((item) => (
          <li key={item.id}>{item.name}</li>
        ))}
      </ul>
    )}
  </div>
);
};

```

Patrón: Efecto Condicional

```

const ConditionalEffect: React.FC<{ enabled: boolean; userId: number }> = ({
  enabled,
  userId,
}) => {
  const [data, setData] = useState(null);

  useEffect(() => {
    // No ejecutar si está deshabilitado
    if (!enabled) {
      return;
    }

    const fetchData = async () => {
      const response = await fetch(`/api/users/${userId}`);
      setData(await response.json());
    };

    fetchData();
  }, [enabled, userId]); // enabled como dependencia

```

```
    return <div>{data ? JSON.stringify(data) : 'Sin datos'}</div>;
  };
}
```

Patrón: Sincronizar con Prop

```
// Cuando necesitas "resetear" estado basado en prop
const EditableField: React.FC<{ initialValue: string; id: string }> = ({
  initialValue,
  id,
}) => {
  const [value, setValue] = useState(initialValue);

  // Sincronizar cuando cambia el ID (nuevo item)
  useEffect(() => {
    setValue(initialValue);
  }, [id]); // Solo cuando cambia id, no initialValue

  return (
    <input
      value={value}
      onChange={(e) => setValue(e.target.value)}
    />
  );
};
```

7. Errores Comunes y Soluciones

Error 1: Dependencias Faltantes

```
// ❌ MAL: Falta dependencia
const BadComponent: React.FC<{ userId: number }> = ({ userId }) => {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then((r) => r.json())
      .then(setUser);
  }, []); // ⚠️ Warning: falta userId en dependencias

  return <div>{user?.name}</div>;
};

// ✅ BIEN: Incluir todas las dependencias
const GoodComponent: React.FC<{ userId: number }> = ({ userId }) => {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then((r) => r.json())
      .then(setUser);
  }, [userId]); // ✅ Correcto
}
```

```
    return <div>{user?.name}</div>;  
  };  
};
```

Error 2: Loop Infinito

```
// ❌ MAL: Objeto como dependencia causa loops  
const LoopComponent: React.FC = () => {  
  const [data, setData] = useState([]);  
  const options = { limit: 10 }; // Nuevo objeto cada render  
  
  useEffect(() => {  
    fetch(`/api/items?limit=${options.limit}`)  
      .then((r) => r.json())  
      .then(setData);  
  }, [options]); // ⚠️ Loop infinito: options siempre es nuevo  
  
  return (  
    <ul>  
      {data.map((item) => (  
        <li key={item.id}>{item.name}</li>  
      ))}  
    </ul>  
  );  
};  
};  
  
// ✅ BIEN: Usar useMemo o valores primitivos  
const FixedComponent: React.FC = () => {  
  const [data, setData] = useState([]);  
  const limit = 10; // Primitivo, no cambia  
  
  // O usar useMemo para objetos  
  const options = useMemo(() => ({ limit: 10 }), []);  
  
  useEffect(() => {  
    fetch(`/api/items?limit=${limit}`)  
      .then((r) => r.json())  
      .then(setData);  
  }, [limit]); // ✅ Sin loop  
  
  return (  
    <ul>  
      {data.map((item) => (  
        <li key={item.id}>{item.name}</li>  
      ))}  
    </ul>  
  );  
};  
};
```

Error 3: Estado después de Unmount

```
// ❌ MAL: Puede causar memory leak  
const LeakyComponent: React.FC = () => {  
  const [data, setData] = useState(null);
```

```

useEffect(() => {
  fetch('/api/slow-data')
    .then((r) => r.json())
    .then(setData); // ⚠ Si se desmonta antes, error
}, []);

return <div>{data}</div>;
};

// ✅ BIEN: Verificar si sigue montado
const SafeComponent: React.FC = () => {
  const [data, setData] = useState(null);

  useEffect(() => {
    let isMounted = true;

    fetch('/api/slow-data')
      .then((r) => r.json())
      .then((result) => {
        if (isMounted) {
          setData(result);
        }
      });

    return () => {
      isMounted = false;
    };
  }, []);

  return <div>{data}</div>;
};

```

8. Hook useLifecycle (Custom)

```

// =====
// QUÉ: Hook useLifecycle para debugging del ciclo de vida
// PARA: Visualizar mount/update/unmount durante desarrollo
// IMPACTO: Facilita entender cuándo se ejecuta cada fase
// =====

import { useEffect, useRef } from 'react';

interface LifecycleCallbacks {
  onMount?: () => void;
  onUpdate?: () => void;
  onUnmount?: () => void;
}

const useLifecycle = (
  componentName: string,
  callbacks?: LifecycleCallbacks
): void => {
  const renderCount = useRef(0);
  const isFirstRender = useRef(true);

```

```

// Mount
useEffect(() => {
  console.log(`[${componentName}] ● Montado`);
  callbacks?.onMount?.();

  return () => {
    console.log(`[${componentName}] ● Desmontado`);
    callbacks?.onUnmount?.();
  };
}, [componentName, callbacks]);

// Update
useEffect(() => {
  renderCount.current += 1;

  if (isFirstRender.current) {
    isFirstRender.current = false;
    return;
  }

  console.log(`[${componentName}] ↻ Actualizado (render #${renderCount.current})`);
  callbacks?.onUpdate?.();
});

export { useLifecycle };

// Uso
const DebugComponent: React.FC = () => {
  useLifecycle('DebugComponent', {
    onMount: () => console.log('Callback de mount'),
    onUpdate: () => console.log('Callback de update'),
    onUnmount: () => console.log('Callback de unmount')
  });

  return <div>Debug</div>;
};

```

Recursos Adicionales

- [React Docs: useEffect](#)
- [React Docs: useLayoutEffect](#)
- [Dan Abramov: A Complete Guide to useEffect](#)

Checklist de Comprensión

- ☐ Conozco las 3 fases del ciclo de vida
- ☐ Sé mapear métodos de clase a hooks
- ☐ Entiendo la diferencia entre useEffect y useLayoutEffect
- ☐ Conozco el orden de ejecución de efectos
- ☐ Sé evitar loops infinitos y memory leaks

Siguiente: [04-vite-configuracion.md](#)

Configuración de Vite

Objetivos

- Entender la arquitectura de Vite
- Configurar vite.config.ts para proyectos React
- Optimizar desarrollo y producción
- Usar variables de entorno correctamente

1. ¿Qué es Vite?



Vite (pronunciado "vit", francés para "rápido") es un build tool moderno que ofrece:

- ⚡ **Servidor de desarrollo ultrarrápido** con Hot Module Replacement (HMR)
- 📦 **Build optimizado** con Rollup
- 🛠️ **Configuración mínima** pero extensible
- 📝 **Soporte nativo para TypeScript**

Comparación con Create React App

Característica	Vite	Create React App
Tiempo de inicio	< 1 segundo	10-30 segundos
HMR	Instantáneo	1-5 segundos
Configuración	vite.config.ts	Eject necesario
Build tool	Rollup (ESBuild)	Webpack
Tamaño bundle	Más pequeño	Más grande

2. Estructura de Proyecto Vite + React

```
mi-proyecto/
├── public/                # Archivos estáticos (no procesados)
│   ├── favicon.ico
│   └── robots.txt
├── src/
│   ├── assets/           # Importados en código (procesados)
│   │   └── logo.svg
│   ├── components/
│   ├── hooks/
│   ├── App.tsx
│   ├── main.tsx          # Punto de entrada
│   └── vite-env.d.ts     # Tipos de Vite
├── index.html            # ← En la raíz (no en public)
├── package.json
├── tsconfig.json
├── tsconfig.node.json    # Para vite.config.ts
└── vite.config.ts        # Configuración de Vite
```

index.html en la raíz

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <link
      rel="icon"
      type="image/svg+xml"
      href="/vite.svg" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1.0" />
    <title>Mi App React</title>
  </head>
  <body>
    <div id="root"></div>
    <!-- Vite inyecta el script automáticamente -->
    <script
      type="module"
      src="/src/main.tsx"></script>
  </body>
</html>
```

3. vite.config.ts Básico

```
// =====
// QUÉ: Configuración mínima de Vite para React
// PARA: Iniciar un proyecto React con TypeScript rápidamente
// IMPACTO: Servidor de desarrollo funcionando en segundos
// =====

import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  // Plugin de React (requerido)
  plugins: [react()],
});
```

4. Configuración Completa

```
// =====
// QUÉ: Configuración avanzada de Vite con todas las opciones
// PARA: Proyectos de producción con alias, proxy, optimizaciones
// IMPACTO: Control total sobre desarrollo y build
// =====

import { defineConfig, loadEnv } from 'vite';
import react from '@vitejs/plugin-react';
import path from 'path';
```

```
export default defineConfig(({ command, mode }) => {
  // Cargar variables de entorno
  const env = loadEnv(mode, process.cwd(), '');

  return {
    // 1 PLUGINS
    plugins: [
      react({
        // Opciones de Fast Refresh
        fastRefresh: true,
        // JSX runtime automático (React 17+)
        jsxRuntime: 'automatic',
      }),
    ],

    // 2 ALIAS DE RUTAS
    resolve: {
      alias: {
        '@': path.resolve(__dirname, './src'),
        '@components': path.resolve(__dirname, './src/components'),
        '@hooks': path.resolve(__dirname, './src/hooks'),
        '@utils': path.resolve(__dirname, './src/utils'),
        '@assets': path.resolve(__dirname, './src/assets'),
        '@types': path.resolve(__dirname, './src/types'),
      },
    },

    // 3 SERVIDOR DE DESARROLLO
    server: {
      port: 3000,
      open: true, // Abrir navegador automáticamente
      cors: true,
      strictPort: true, // Fallar si puerto ocupado

      // Proxy para API (evitar CORS en desarrollo)
      proxy: {
        '/api': {
          target: env.VITE_API_URL || 'http://localhost:8080',
          changeOrigin: true,
          rewrite: (path) => path.replace(/^\/api/, ''),
        },
      },

      // Watch para archivos extra
      watch: {
        usePolling: true, // Útil en Docker
      },
    },

    // 4 BUILD DE PRODUCCIÓN
    build: {
      outDir: 'dist',
      sourcemap: mode !== 'production',
      minify: 'terser',
    },
  };
});
```



```

// Chunk splitting
rollupOptions: {
  output: {
    manualChunks: {
      // Separar vendors grandes
      vendor: ['react', 'react-dom'],
      // Separar librerías de UI
      // ui: ['@headlessui/react', 'framer-motion'],
    },
  },
},

// Tamaño de chunks
chunkSizeWarningLimit: 500,
},

// 5 PREVIEW (sirve build de producción)
preview: {
  port: 4173,
  open: true,
},

// 6 OPTIMIZACIÓN DE DEPENDENCIAS
optimizeDeps: {
  include: ['react', 'react-dom'],
  // Excluir si causan problemas
  // exclude: ['some-package'],
},

// 7 CSS
css: {
  modules: {
    localsConvention: 'camelCase',
  },
  preprocessorOptions: {
    scss: {
      additionalData: `@import "@styles/variables.scss";`,
    },
  },
},

// 8 DEFINICIÓN DE CONSTANTES GLOBALES
define: {
  __APP_VERSION__: JSON.stringify(process.env.npm_package_version),
},

// 9 ESBUILD (transpilación)
esbuild: {
  // Eliminar console.log en producción
  drop: mode === 'production' ? ['console', 'debugger'] : [],
},
};
});

```

5. Alias de Rutas con TypeScript

Para que TypeScript reconozca los alias, actualizar `tsconfig.json` :

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"],
      "@components/*": ["src/components/*"],
      "@hooks/*": ["src/hooks/*"],
      "@utils/*": ["src/utils/*"],
      "@assets/*": ["src/assets/*"],
      "@types/*": ["src/types/*"]
    }
  }
}
```

Uso de Alias

```
// ❌ Sin alias
import { Button } from '../../components/ui/Button';
import { useAuth } from '../../hooks/useAuth';

// ✅ Con alias
import { Button } from '@components/ui/Button';
import { useAuth } from '@hooks/useAuth';
```

6. Variables de Entorno

Archivos de Entorno

```
mi-proyecto/
├── .env                # Siempre cargado
├── .env.local          # Siempre cargado, ignorado por git
├── .env.development    # Solo en dev
├── .env.development.local
├── .env.production     # Solo en build
└── .env.production.local
```

Prefijo VITE_

```
# .env
# ❌ No accesible en cliente (sin prefijo)
SECRET_KEY=mi-secreto

# ✅ Accesible en cliente
VITE_API_URL=https://api.example.com
VITE_APP_TITLE=Mí Aplicación
VITE_ENABLE_ANALYTICS=true
```

Uso en Código

```
// =====  
// QUÉ: Acceso a variables de entorno en React con Vite  
// PARA: Configurar URLs de API, feature flags, etc.  
// IMPACTO: Diferentes configuraciones por ambiente (dev/prod)  
// =====  
  
// Acceder a variables  
const apiUrl = import.meta.env.VITE_API_URL;  
const appTitle = import.meta.env.VITE_APP_TITLE;  
  
// Variables especiales de Vite  
const isDev = import.meta.env.DEV; // true en desarrollo  
const isProd = import.meta.env.PROD; // true en producción  
const mode = import.meta.env.MODE; // 'development' | 'production'  
const baseUrl = import.meta.env.BASE_URL; // ''  
  
// Ejemplo en componente  
const ApiConfig: React.FC = () => {  
  return (  
    <div>  
      <p>API: {import.meta.env.VITE_API_URL}</p>  
      <p>Modo: {import.meta.env.MODE}</p>  
      {import.meta.env.DEV && <p>⚠ Modo desarrollo</p>}  
    </div>  
  );  
};
```

Tipos para Variables de Entorno

```
// src/vite-env.d.ts  
/// <reference types="vite/client" />  
  
interface ImportMetaEnv {  
  readonly VITE_API_URL: string;  
  readonly VITE_APP_TITLE: string;  
  readonly VITE_ENABLE_ANALYTICS: string;  
  // Agregar más variables según necesites  
}  
  
interface ImportMeta {  
  readonly env: ImportMetaEnv;  
}
```

7. Scripts de Package.json

```
{  
  "scripts": {  
    "dev": "vite",  
    "build": "tsc && vite build",  
    "preview": "vite preview",
```

```

"lint": "eslint . --ext ts,tsx --report-unused-disable-directives --max-warnings 0",
"type-check": "tsc --noEmit",

// Modos personalizados: "",
"dev:staging": "vite --mode staging",
"build:staging": "tsc && vite build --mode staging",

// Análisis: "",
"build:analyze": "vite build --mode production && npx vite-bundle-visualizer"
}
}

```

8. Proxy para APIs

```

// =====
// QUÉ: Configuración de proxy para APIs en desarrollo
// PARA: Evitar errores CORS al llamar a APIs externas
// IMPACTO: Desarrollo local sin configurar CORS en el backend
// =====

export default defineConfig({
  server: {
    proxy: {
      // Proxy simple
      '/api': 'http://localhost:8080',

      // Proxy con opciones
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
        secure: false,
        rewrite: (path) => path.replace(/^\/api/, ''),
        configure: (proxy, options) => {
          proxy.on('proxyReq', (proxyReq, req, res) => {
            console.log('Proxy request:', req.url);
          });
        },
      },
    },

    // WebSocket proxy
    '/socket.io': {
      target: 'ws://localhost:8080',
      ws: true,
    },
  },
});

```

Uso en Código

```

// En desarrollo: /api/users → http://localhost:8080/users
// En producción: /api/users → tu-dominio.com/api/users

```

```
const fetchUsers = async () => {
  // Usa ruta relativa, el proxy maneja el resto
  const response = await fetch('/api/users');
  return response.json();
};
```

9. Importación de Assets

Imágenes y SVG

```
// Importación directa (incluido en bundle)
import logo from '@assets/logo.svg';
import heroImage from '@assets/hero.png';

const Header: React.FC = () => (
  <header>
    <img
      src={logo}
      alt="Logo"
    />
    <img
      src={heroImage}
      alt="Hero"
    />
  </header>
);

// SVG como componente React
import { ReactComponent as Logo } from '@assets/logo.svg'; // ❌ No funciona en Vite

// ✅ Usar vite-plugin-svgr para SVG como componentes
import Logo from '@assets/logo.svg?react';

const Header: React.FC = () => (
  <header>
    <Logo className="logo" />
  </header>
);
```

URL de Assets

```
// Obtener URL del asset
const imageUrl = new URL('./image.png', import.meta.url).href;

// Assets en carpeta public (no procesados)
const publicImage = '/images/hero.jpg'; // → public/images/hero.jpg
```

10. Configuración para Docker

```
// =====
// QUÉ: Configuración de Vite para contenedores Docker
```

```
// PARA: Desarrollo con hot-reload dentro de contenedores
// IMPACTO: Entorno de desarrollo consistente entre equipos
// =====

export default defineConfig({
  server: {
    host: '0.0.0.0', // Accesible desde fuera del contenedor
    port: 3000,
    watch: {
      usePolling: true, // Necesario para Docker en algunos sistemas
    },
  },
});
```

Dockerfile Multi-Stage

```
# Stage 1: Build
FROM node:20-alpine AS builder

WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN corepack enable && pnpm install --frozen-lockfile

COPY . .
RUN pnpm build

# Stage 2: Serve con Nginx
FROM nginx:alpine AS production

COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

11. Plugins Útiles

```
// =====
// QUÉ: Plugins recomendados para Vite + React + TypeScript
// PARA: Extender funcionalidad (SVG, type-checking, etc.)
// IMPACTO: Mejor DX con validación en tiempo real
// =====

import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import svgr from 'vite-plugin-svgr';
import checker from 'vite-plugin-checker';

export default defineConfig({
  plugins: [
    // React con Fast Refresh
    react(),
```

```
// SVG como componentes React
svgr({
  svgrOptions: {
    icon: true,
  },
}),

// Type checking en tiempo real
checker({
  typescript: true,
  eslint: {
    lintCommand: 'eslint "./src/**/*.ts,tsx"',
  },
}),
],
});
```

Instalación de Plugins

```
# Instalar plugins
pnpm add -D vite-plugin-svgr vite-plugin-checker
```



Recursos Adicionales

- [Vite Documentation](#)
- [Vite Plugin React](#)
- [Awesome Vite](#)



Checklist de Comprensión

- ☐ Entiendo la estructura de un proyecto Vite
- ☐ Sé configurar alias de rutas
- ☐ Conozco el sistema de variables de entorno
- ☐ Puedo configurar un proxy para APIs
- ☐ Sé optimizar el build de producción

Final de la teoría · Semana 06



Ejercicio 01: Hooks Básicos



Objetivos

- Crear hooks simples con estado boolean
- Implementar `useToggle` y `useCounter`
- Aprender a tipar retornos de hooks
- Entender `useCallback` para funciones estables



Descripción

En este ejercicio crearás tus primeros custom hooks básicos que encapsulan patrones de estado comunes.

Duración estimada

30 minutos

Conceptos cubiertos

- useState en custom hooks
- useCallback para memoización
- Interfaces para tipos de retorno
- Parámetros opcionales con defaults

Instrucciones

Paso 1: Crear useToggle

Abre `starter/useToggle.ts` y descomenta el código para crear un hook que:

- Maneje un estado boolean
- Tenga funciones: `toggle`, `setTrue`, `setFalse`
- Reciba un valor inicial opcional

Paso 2: Crear useCounter

Abre `starter/useCounter.ts` y descomenta el código para crear un hook que:

- Maneje un contador numérico
- Tenga funciones: `increment`, `decrement`, `reset`
- Soporte opciones: `min`, `max`, `step`

Paso 3: Probar los hooks

Abre `starter/App.tsx` y descomenta el código para probar ambos hooks en componentes de ejemplo.

Criterios de evaluación

Criterio	Puntos
useToggle funciona correctamente	25
useCounter con opciones	35
Tipado TypeScript completo	25
useCallback en funciones	15
Total	100

Archivos

```
ejercicio-01-hooks-basicos/  
├─ README.md  
├─ starter/  
│   ├─ useToggle.ts  
│   ├─ useCounter.ts  
│   └─ App.tsx  
└─ solution/
```



```
├─ useToggle.ts
├─ useCounter.ts
└─ App.tsx
```

Pistas

- `useCallback` evita que las funciones se creen en cada render
- El array de dependencias vacío `[]` significa que la función nunca cambia
- Usa desestructuración con defaults para opciones opcionales

Siguiente: [Ejercicio 02 - Hooks con Estado](#)



Ejercicio 02: Hooks con Estado Complejo



Objetivos

- Crear hooks que manejan objetos y arrays
- Implementar `useForm` para formularios
- Implementar `useArray` para listas
- Manejar estado complejo de forma inmutable



Descripción

Crearás hooks que manejan estructuras de datos más complejas que primitivos.



Duración estimada

40 minutos



Conceptos cubiertos

- Estado con objetos tipados
- Operaciones inmutables en arrays
- Generics para tipado flexible
- Validación en tiempo real



Instrucciones

Paso 1: Crear `useForm`

Un hook para manejar formularios con:

- Estado de valores
- Función `handleChange`
- Función `reset`
- Validación básica

Paso 2: Crear `useArray`

Un hook para manejar arrays con:

- Operaciones CRUD (`push`, `remove`, `update`)
- Filtrado

- Clear

✓ Criterios de evaluación

Criterio	Puntos
useForm funcional	30
useArray con todas las ops	35
TypeScript genérico	20
Inmutabilidad correcta	15
Total	100

Siguiente: [Ejercicio 03 - Hooks con Efectos](#)



Ejercicio 03: Hooks con Efectos



Objetivos

- Crear hooks que usen useEffect
- Implementar useFetch para llamadas API
- Implementar useDebounce para optimización
- Manejar cleanup correctamente



Duración estimada

45 minutos



Conceptos cubiertos

- useEffect con dependencias
- Cleanup y cancelación
- AbortController para fetch
- Debounce pattern



Instrucciones

Paso 1: Crear useFetch

Hook que:

- Hace fetch a una URL
- Maneja estados loading/error/data
- Incluye refetch
- Cancela requests pendientes

Paso 2: Crear useDebounce

Hook que:

- Retrasa la actualización de un valor
- Útil para búsquedas en tiempo real

✓ Criterios de evaluación

Criterio	Puntos
useFetch con todos los estados	35
Cancelación con AbortController	20
useDebounce funcional	25
Cleanup correcto	20
Total	100

Siguiente: [Ejercicio 04 - Hooks Genéricos](#)

Ejercicio 04: Hooks Genéricos

🎯 Objetivo

Implementar custom hooks utilizando **generics de TypeScript** para crear hooks reutilizables y type-safe que funcionen con cualquier tipo de dato.

📖 Conceptos Cubiertos

- Generics en custom hooks
- Type inference con generics
- Serialización/deserialización tipada
- Hooks para operaciones asíncronas genéricas
- Patrones de TypeScript avanzados

📁 Estructura

```
ejercicio-04-hooks-genericos/  
├── README.md  
├── starter/  
│   ├── useLocalStorage.ts    # Hook genérico para localStorage  
│   ├── useAsync.ts           # Hook genérico para operaciones async  
│   └── App.tsx                # Demo de uso  
└── solution/  
    ├── useLocalStorage.ts  
    ├── useAsync.ts  
    └── App.tsx
```

🕒 Duración Estimada

45 minutos

📋 Hooks a Implementar

useLocalStorage

Hook genérico para persistir estado en localStorage con serialización automática.

Características:

- Genérico para cualquier tipo serializable
- Serialización/deserialización automática con JSON
- Valor inicial con lazy initialization
- Manejo de errores de parsing
- Sincronización entre pestañas (storage event)

Interface:

```
interface UseLocalStorageReturn<T> {  
  value: T;  
  setValue: (value: T | ((prev: T) => T)) => void;  
  removeValue: () => void;  
}
```

useAsync

Hook genérico para manejar operaciones asíncronas con estados de loading, error y data.

Características:

- Acepta cualquier función que retorne Promise
- Estados: idle, loading, success, error
- Función execute para disparar la operación
- Reiniciar estado con reset

Interface:

```
type AsyncStatus = 'idle' | 'loading' | 'success' | 'error';  
  
interface UseAsyncReturn<T> {  
  data: T | null;  
  status: AsyncStatus;  
  error: Error | null;  
  isLoading: boolean;  
  isError: boolean;  
  isSuccess: boolean;  
  execute: (...args: unknown[]) => Promise<T | null>;  
  reset: () => void;  
}
```

Instrucciones

Paso 1: useLocalStorage (20 min)

1. Abre `starter/useLocalStorage.ts`
2. Lee los comentarios explicativos
3. Descomenta el código de la interfaz
4. Descomenta la implementación del hook
5. Observa el uso de generics `<T>`

Paso 2: useAsync (15 min)

1. Abre `starter/useAsync.ts`
2. Implementa los tipos de estado
3. Descomenta el hook genérico

4. Observa cómo el tipo `T` fluye a través del hook

Paso 3: Demo en App (10 min)

1. Abre `starter/App.tsx`
2. Descomenta los demos
3. Observa la inferencia de tipos automática
4. Prueba diferentes tipos de datos

✓ Criterios de Éxito

Criterio	Peso
localStorage funciona con diferentes tipos	30%
Serialización/deserialización correcta	20%
useAsync maneja todos los estados	25%
TypeScript infiere tipos correctamente	15%
Cleanup y manejo de errores	10%

💡 Tips

- Los generics permiten que TypeScript infiera el tipo del valor inicial
- Usa `JSON.stringify` y `JSON.parse` para serialización
- Envuelve operaciones de localStorage en try/catch
- El tipo `T` en `useState<T>` debe coincidir con el genérico del hook

🔗 Recursos

- [TypeScript Generics](#)
- [React + TypeScript Generics](#)

Ejercicio 05: Hooks Compuestos

🎯 Objetivo

Aprender a **componer hooks** combinando múltiples hooks más simples para crear abstracciones de alto nivel que encapsulan lógica de negocio compleja.

📖 Conceptos Cubiertos

- Composición de hooks (usar hooks dentro de otros hooks)
- Abstracción de lógica de negocio
- Patrones de autenticación con hooks
- Encapsulación de efectos secundarios
- Hooks de feature completa

📁 Estructura

```
ejercicio-05-hooks-compuestos/  
├── README.md  
├── starter/  
│   └── hooks/
```

```

|   |   | useLocalStorage.ts # Hook base (del ejercicio anterior)
|   |   | useFetch.ts       # Hook base
|   |   | useAuth.ts        # Hook compuesto
|   |   | types.ts          # Tipos compartidos
|   |   | App.tsx           # Demo de uso
|   |   |
|   |   | solution/
|   |   |   | hooks/
|   |   |   |   | useLocalStorage.ts
|   |   |   |   | useFetch.ts
|   |   |   |   | useAuth.ts
|   |   |   | types.ts
|   |   |   | App.tsx

```

Duración Estimada

50 minutos

Hook a Implementar

useAuth (Hook Compuesto)

Hook que combina `useLocalStorage` + lógica de autenticación para proporcionar una abstracción completa de auth.

Composición:

```

useAuth
| useLocalStorage (para token y user)
| useState (para loading state)
| useCallback (para funciones memoizadas)
| useEffect (para verificar token al montar)

```

Características:

- Login con credenciales
- Logout con limpieza de storage
- Persistencia de sesión entre recargas
- Estado de autenticación derivado
- Verificación automática de token

Interface:

```

interface UseAuthReturn {
  user: User | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  error: string | null;
  login: (email: string, password: string) => Promise<boolean>;
  logout: () => void;
  clearError: () => void;
}

```

Instrucciones

Paso 1: Revisar hooks base (10 min)

1. Revisa `starter/hooks/useLocalStorage.ts`
2. Revisa `starter/hooks/useFetch.ts`

3. Entiende cómo funcionan individualmente

Paso 2: Implementar useAuth (25 min)

1. Abre `starter/hooks/useAuth.ts`
2. Observa cómo se importan los hooks base
3. Descomenta y entiende cada sección:
 - Estado con `useLocalStorage`
 - Estados derivados
 - Función login
 - Función logout
 - Efecto de verificación

Paso 3: Demo en App (15 min)

1. Abre `starter/App.tsx`
2. Descomenta el formulario de login
3. Observa cómo el hook encapsula toda la lógica
4. Prueba login/logout

✓ Criterios de Éxito

Criterio	Peso
useAuth utiliza hooks base correctamente	25%
Login funciona con persistencia	25%
Logout limpia estado y storage	20%
Estados derivados correctos	15%
Manejo de errores	15%

💡 Tips

- Los hooks compuestos son como "feature hooks" que encapsulan funcionalidad completa
- Cada hook base debe ser independiente y testeable
- El hook compuesto coordina los hooks base sin duplicar su lógica
- Los estados derivados (`isAuthenticated`) deben calcularse a partir del estado base

🔗 Recursos

- [React Hooks Composition Patterns](#)
- [Authentication Patterns with Hooks](#)

🎯 Proyecto Integrador: Dashboard Interactivo

📋 Descripción

Este es el **proyecto final de la Etapa 2 (Fundamentos de React)** del bootcamp. Construirás un **Dashboard Interactivo** que integre todos los conceptos aprendidos durante las semanas 2-6.

El dashboard debe demostrar dominio de:

- Custom hooks tipados
- Composición de componentes
- Context API para estado global

- TypeScript estricto
- Patrones de React modernos

Tu Dominio Asignado

El instructor te asignará un dominio único. Algunos ejemplos:

Dominio	Widgets de ejemplo
 Biblioteca	Libros prestados, lectores activos, devoluciones pendientes
 Farmacia	Medicamentos bajo stock, ventas del día, recetas pendientes
 Gimnasio	Miembros activos, clases del día, equipos en mantenimiento
 Escuela	Asistencia, calificaciones, tareas pendientes
 Restaurante	Pedidos activos, mesas ocupadas, inventario bajo
 Hospital	Pacientes en espera, camas disponibles, citas del día

Objetivos del Proyecto

1. **Implementar 5+ custom hooks** aplicados a tu dominio
2. **Crear 3+ widgets** que muestren información relevante
3. **Usar Context API** para preferencias del usuario
4. **Aplicar TypeScript estricto** en todo el proyecto
5. **Organizar código** siguiendo arquitectura escalable

Estructura del Proyecto

```
3-proyecto/
├─ README.md                # Este archivo
├─ starter/
│  └─ src/
│     │
│     │   └─ hooks/          # Custom hooks
│     │       │
│     │       │   └─ useLocalStorage.ts
│     │       │   └─ useFetch.ts
│     │       │   └─ useToggle.ts
│     │       └─ index.ts    # Barrel export
│     │
│     │   └─ context/
│     │       │
│     │       │   └─ ThemeContext.tsx
│     │       └─ index.ts
│     │
│     │   └─ components/
│     │       │
│     │       │   └─ widgets/      # Widgets del dashboard
│     │       │       │
│     │       │       │   └─ README.md  # Instrucciones
│     │       │       └─ layout/
│     │       │           │
│     │       │           │   └─ Header.tsx
│     │       │           │   └─ Sidebar.tsx
│     │       │           └─ ui/
│     │       │               │
│     │       │               │   └─ Card.tsx
│     │       │               │   └─ Button.tsx
│     │       │               └─ Spinner.tsx
│     │       └─ types/
│     │           └─ index.ts
│     └─ App.tsx
└─ main.tsx
```



```
| |— index.html
| |— package.json
| |— tsconfig.json
| |— vite.config.ts
|— solution/                                # Solución de referencia (genérica)
```

✓ Requisitos Funcionales

1. Custom Hooks (obligatorios)

Debes implementar **al menos 5** custom hooks:

Hook	Descripción	Uso en dashboard
useLocalStorage<T>	Persistencia de preferencias	Tema, sidebar collapsed
useFetch<T>	Llamadas a API	Cargar datos del dominio
useToggle	Boolean toggle	Sidebar, modals
useDebounce	Optimización	Búsqueda en tiempo real
useFilter<T>	Filtrar arrays	Filtrar items del dominio

2. Widgets (mínimo 3)

Cada widget debe:

- Mostrar datos relevantes del dominio
- Usar al menos un custom hook
- Tener estados de loading/error
- Ser visualmente distintivo

Ejemplos por dominio:

- **Biblioteca:** Libros más prestados, Préstamos vencidos, Nuevas adquisiciones
- **Farmacia:** Medicamentos bajo stock, Ventas por categoría, Alertas de caducidad
- **Gimnasio:** Check-ins del día, Clases populares, Membresías por vencer

3. Context API

Implementar `ThemeContext` con:

- Tema claro/oscuro
- Colores primarios configurables
- Persistencia en localStorage

4. Layout Responsive

- Header con título y controles
- Sidebar colapsable
- Grid de widgets adaptable

🔧 Instrucciones Paso a Paso

Paso 1: Setup (10 min)

```
cd 3-proyecto/starter
pnpm install
```

pnpm dev

Paso 2: Implementar Hooks (30 min)

- 1. Completa los hooks en `src/hooks/`
- 2. Verifica que los tipos sean correctos
- 3. Exporta desde `index.ts`

Paso 3: Configurar Context (20 min)

- 1. Implementa `ThemeContext` con provider
- 2. Agrega tipos para el tema
- 3. Conecta con `useLocalStorage`

Paso 4: Crear Widgets (40 min)

- 1. Define tipos para tu dominio en `src/types/`
- 2. Crea 3 widgets en `src/components/widgets/`
- 3. Usa hooks para data fetching
- 4. Aplica estilos del tema

Paso 5: Integrar Dashboard (20 min)

- 1. Configura el layout en `App.tsx`
- 2. Renderiza los widgets en grid
- 3. Conecta sidebar y header
- 4. Prueba responsive design

 Rúbrica de Evaluación

Criterio	Peso	Descripción
Custom Hooks	25%	5+ hooks funcionales y tipados
Widgets	25%	3+ widgets con loading/error states
Context API	15%	Tema funcional con persistencia
TypeScript	15%	Tipos estrictos, sin any
UI/UX	10%	Diseño coherente y responsive
Código Limpio	10%	Organización, naming, comentarios

Niveles de Logro

- **Excelente (90-100%):** Supera requisitos, hooks adicionales, animaciones
- **Bueno (70-89%):** Cumple todos los requisitos base
- **Suficiente (50-69%):** Cumple requisitos mínimos con deficiencias
- **Insuficiente (<50%):** No cumple requisitos mínimos

 Tips y Mejores Prácticas

TypeScript

```
// ✅ Usar generics para hooks reutilizables
const useFetch = <T,>(url: string): UseFetchReturn<T> => { ... };

// ✅ Interfaces para props
interface WidgetProps {
```

```
title: string;
data: DomainItem[];
onRefresh?: () => void;
}

// ❌ Evitar any
const data: any = response; // NO
```

Hooks

```
// ✅ Composición de hooks
const useWidgetData = (endpoint: string) => {
  const { data, loading } = useFetch<Item[]>(endpoint);
  const [filtered, setFiltered] = useState<Item[]>([]);
  const debouncedFilter = useDebounce(filterTerm, 300);
  // ...
};
```

Context

```
// ✅ Separar lógica en custom hook
const useTheme = () => {
  const context = useContext(ThemeContext);
  if (!context) throw new Error('useTheme debe usarse dentro de ThemeProvider');
  return context;
};
```



Recursos

- [React Custom Hooks](#)
- [Context API](#)
- [TypeScript Generics](#)



Tiempo Estimado

Total: 2-2.5 horas

- Setup: 10 min
- Hooks: 30 min
- Context: 20 min
- Widgets: 40 min
- Integración: 20 min
- Testing/Pulido: 20 min



Entregables

1. Código funcional en `starter/`
 2. README actualizado con:
 - Tu dominio asignado
 - Descripción de widgets implementados
 - Screenshots del dashboard
 3. Sin errores de TypeScript
 4. Aplicación ejecutable con `pnpm dev`
-

🎨 Widgets del Dashboard

📋 Instrucciones

Crea **al menos 3 widgets** para tu dominio asignado. Cada widget debe:

- 1. Usar un componente `Card` como contenedor
- 2. Mostrar datos relevantes del dominio
- 3. Usar al menos un custom hook (`useFetch` , `useFilter` , etc.)
- 4. Manejar estados de loading y error
- 5. Aplicar estilos del tema

📁 Estructura Sugerida

```
widgets/
├── README.md           # Este archivo
├── index.ts            # Barrel export
├── StatsWidget.tsx     # Estadísticas generales
├── ListWidget.tsx      # Lista de items
├── AlertWidget.tsx     # Alertas/notificaciones
└── ...                 # Widgets adicionales
```

🏛 Ejemplos por Dominio

📖 Biblioteca

Widget	Descripción	Hooks usados
BooksStats	Libros totales, disponibles, prestados	useFetch
RecentLoans	Últimos préstamos realizados	useFetch, useFilter
DueAlerts	Préstamos por vencer	useFetch

💊 Farmacia

Widget	Descripción	Hooks usados
StockAlerts	Medicamentos bajo stock	useFetch
TodaySales	Ventas del día	useFetch
ExpiringMeds	Próximos a caducar	useFetch, useFilter

🏋 Gimnasio

Widget	Descripción	Hooks usados
DailyCheckins	Check-ins de hoy	useFetch
PopularClasses	Clases más populares	useFetch
ExpiringMemberships	Membresías por vencer	useFetch

Template de Widget

```
import React from 'react';
import { useFetch } from '@/hooks';
import { Card } from '@/components/ui/Card';
import { Spinner } from '@/components/ui/Spinner';
import type { WidgetProps, DomainItem } from '@/types';

interface MyWidgetProps extends WidgetProps {
  // Props específicas del widget
}

export const MyWidget: React.FC<MyWidgetProps> = ({ title }) => {
  const { data, loading, error, refetch } =
    useFetch<DomainItem[]>('/api/items');

  if (loading) {
    return (
      <Card title={title}>
        <Spinner />
      </Card>
    );
  }

  if (error) {
    return (
      <Card title={title}>
        <p style={{ color: 'red' }}>Error: {error.message}</p>
        <button onClick={refetch}>Reintentar</button>
      </Card>
    );
  }

  return (
    <Card
      title={title}
      onRefresh={refetch}>
      {data?.map((item) => (
        <div key={item.id}>{item.name}</div>
      ))}
    </Card>
  );
};
```

Checklist

- ☐ Widget 1: ____ (stats/resumen)
- ☐ Widget 2: ____ (lista/detalle)
- ☐ Widget 3: ____ (alertas/acciones)
- ☐ Todos usan Card como contenedor
- ☐ Todos manejan loading state
- ☐ Todos manejan error state
- ☐ Todos usan al menos un custom hook

Glosario - Semana 06

A

AbortController

Interfaz de la Web API que permite cancelar peticiones fetch y otras operaciones asíncronas. Esencial para evitar memory leaks en hooks de fetching.

```
const controller = new AbortController();
fetch(url, { signal: controller.signal });
controller.abort(); // Cancela la petición
```

B

Barrel Export

Patrón de exportación que consolida múltiples exports en un solo archivo `index.ts`, simplificando las importaciones.

```
// hooks/index.ts
export { useToggle } from './useToggle';
export { useFetch } from './useFetch';
```

C

Cleanup Function

Función retornada por `useEffect` que se ejecuta cuando el componente se desmonta o antes de re-ejecutar el efecto. Esencial para limpiar timers, suscripciones y cancelar requests.

```
useEffect(() => {
  const timer = setInterval(tick, 1000);
  return () => clearInterval(timer); // Cleanup
}, []);
```

Composición de Hooks

Patrón de crear hooks que utilizan otros hooks internamente, permitiendo abstracciones de alto nivel.

```
const useAuth = () => {
  const { value: user } = useLocalStorage('user', null);
  const { execute: login } = useFetch();
  // Combina múltiples hooks
};
```

Custom Hook

Función de JavaScript/TypeScript que comienza con "use" y puede llamar a otros hooks. Permite extraer y reutilizar lógica con estado.

D

Debounce

Técnica de optimización que retrasa la ejecución de una función hasta que pase un tiempo determinado sin nuevas invocaciones.

```
const debouncedValue = useDebounce(searchTerm, 300);
```

Dependency Array

Array de dependencias pasado a `useEffect`, `useCallback`, o `useMemo` que determina cuándo re-ejecutar el hook.

E

ESM (ES Modules)

Sistema de módulos estándar de JavaScript usado por Vite, que permite `import/export` nativos.

G

Generics

Feature de TypeScript que permite crear componentes y funciones reutilizables que trabajan con cualquier tipo.

```
const useLocalStorage = <T,>(key: string, initial: T): T => { ... };
```

H

HMR (Hot Module Replacement)

Característica de Vite que actualiza módulos en el navegador sin recargar toda la página, preservando el estado de la aplicación.

Hook Rules

Reglas que deben seguirse al usar hooks:

1. Solo llamar hooks en el nivel superior
2. Solo llamar hooks desde componentes funcionales o custom hooks

L

Lazy Initialization

Técnica donde `useState` recibe una función en lugar de un valor, ejecutándose solo en el primer render.

```
const [state, setState] = useState(() => {  
  return expensiveComputation();  
});
```

M

Memoization

Técnica de optimización que cachea resultados de funciones para evitar recálculos innecesarios. Implementada con `useMemo` y `useCallback`.

R

Ref

Objeto mutable ({ current: T }) que persiste durante todo el ciclo de vida del componente sin causar re-renders.

```
const timerRef = useRef<NodeJS.Timeout | null>(null);
```

S

Signal (AbortSignal)

Objeto que permite comunicar cuando una operación debe ser abortada. Creado a partir de `AbortController` .

State Derivado

Estado que se calcula a partir de otro estado existente, en lugar de almacenarse por separado.

```
const isAuthenticated = user !== null && token !== null;
```

T

Type Inference

Capacidad de TypeScript de deducir tipos automáticamente sin anotaciones explícitas.

```
const count = useState(0); // TypeScript infiere number
```

U

useCallback

Hook que memoriza una función, retornando la misma referencia entre renders si las dependencias no cambian.

useEffect

Hook para ejecutar efectos secundarios (fetching, suscripciones, DOM manual) después del render.

useMemo

Hook que memoriza un valor calculado, recalculándolo solo cuando las dependencias cambian.

useRef

Hook que retorna un objeto ref mutable que persiste durante el ciclo de vida del componente.

useState

Hook para agregar estado local a componentes funcionales.

V

Vite

Build tool moderno para desarrollo web que usa ESM nativo para desarrollo rápido y Rollup para producción optimizada.

```
pnpm create vite@latest
```

Semana 06: Custom Hooks y Proyecto Integrador

Objetivos de Aprendizaje

Al finalizar esta semana, serás capaz de:

- ✓ Crear custom hooks reutilizables con TypeScript
- ✓ Extraer lógica de componentes a hooks personalizados
- ✓ Aplicar patrones de hooks: composición, encapsulamiento
- ✓ Tipar hooks con generics para máxima flexibilidad
- ✓ Comprender el ciclo de vida de componentes con hooks
- ✓ Optimizar la configuración de Vite para desarrollo
- ✓ Integrar todos los conceptos en un Dashboard interactivo

Requisitos Previos

- ✓ Week-01: Fundamentos de TypeScript
- ✓ Week-02: Introducción a React y JSX/TSX
- ✓ Week-03: useEffect y Efectos Secundarios
- ✓ Week-04: Renderizado Condicional y Listas
- ✓ Week-05: Composición de Componentes y Context API

Estructura de la Semana

```
week-06/
├── README.md                # Este archivo
├── rubrica-evaluacion.md    # Criterios de evaluación
├── 0-assets/                # Diagramas SVG
│   ├── 01-custom-hooks-anatomy.svg
│   ├── 02-hooks-composition.svg
│   ├── 03-component-lifecycle.svg
│   ├── 04-hooks-rules.svg
│   └── 05-vite-architecture.svg
├── 1-teoria/                # Material teórico
│   ├── 01-custom-hooks-fundamentos.md
│   ├── 02-patrones-custom-hooks.md
│   ├── 03-ciclo-vida-componentes.md
│   └── 04-vite-configuracion.md
├── 2-ejercicios/            # Ejercicios guiados
│   ├── ejercicio-01-hooks-basicos/
│   ├── ejercicio-02-hooks-con-estado/
│   ├── ejercicio-03-hooks-con-efectos/
│   ├── ejercicio-04-hooks-genericos/
│   └── ejercicio-05-hooks-compuestos/
├── 3-proyecto/              # Proyecto Integrador Final Etapa 2
│   ├── README.md
│   ├── starter/
│   └── solution/
├── 4-recursos/              # Recursos adicionales
│   ├── ebooks-free/
│   ├── videografia/
│   └── webgrafia/
```

Contenidos

1. Teoría (2-2.5 horas)

Archivo	Tema	Duración
01-custom-hooks-fundamentos.md	Anatomía y creación de hooks	30 min
02-patrones-custom-hooks.md	Patrones y mejores prácticas	35 min
03-ciclo-vida-componentes.md	Ciclo de vida con hooks	30 min
04-vite-configuracion.md	Configuración avanzada de Vite	25 min

2. Ejercicios Guiados (3-3.5 horas)

Ejercicio	Tema	Duración
Ejercicio 01	Hooks básicos (useToggle)	30 min
Ejercicio 02	Hooks con estado complejo	40 min
Ejercicio 03	Hooks con efectos	45 min
Ejercicio 04	Hooks con TypeScript generics	40 min
Ejercicio 05	Composición de hooks	45 min

3. Proyecto Integrador Etapa 2 (2-2.5 horas)

Dashboard Interactivo

Proyecto final de la Etapa 2 que integra TODO lo aprendido:

- Custom hooks para lógica reutilizable
- Context API para estado global (tema, usuario)
- Composición de componentes (Cards, Widgets, Layouts)
- Renderizado condicional y listas dinámicas
- Efectos para fetching de datos
- TypeScript estricto en toda la aplicación
- Configuración optimizada de Vite

Distribución del Tiempo (8 horas)

Actividad	Tiempo
Teoría	2-2.5h
Ejercicios	3-3.5h
Proyecto	2-2.5h
Total	~8h

Entregable

 **Nota:** La teoría y los ejercicios son recursos de aprendizaje para prepararte para el proyecto. No son entregables evaluados.

✅ Preparación (No evaluada)

- ☐ Ejercicio 01: `useToggle` y `useCounter` funcionando
- ☐ Ejercicio 02: `useForm` y `useArray` con estado tipado
- ☐ Ejercicio 03: `useFetch` y `useDebounce` con efectos
- ☐ Ejercicio 04: Hooks genéricos `useLocalStorage<T>` y `useAsync<T>`
- ☐ Ejercicio 05: Composición `useAuth` = `useLocalStorage` + `useFetch`

Proyecto Semanal (100% de la evaluación)

- ☐ Dashboard con al menos 5 custom hooks implementados
- ☐ Mínimo 3 widgets interactivos (estadísticas, lista, formulario)
- ☐ Tema claro/oscuro con Context API
- ☐ Datos mock con simulación de fetch
- ☐ TypeScript estricto sin errores
- ☐ Código limpio y documentado

Criterios de Evaluación

Criterio	Peso
Funcionalidad	40%
Adaptación al Dominio	35%
Calidad del Código	25%

Nota: Se requiere mínimo 70% en el proyecto para aprobar la semana.

Logro Desbloqueado

Al completar esta semana, habrás terminado la **Etapla 2: Fundamentos de React**.

Habilidades adquiridas:

- ✅ Componentes funcionales con TypeScript
- ✅ Props, estado y eventos tipados
- ✅ `useEffect` y manejo de efectos
- ✅ Renderizado condicional y listas
- ✅ Composición y Context API
- ✅ Custom hooks reutilizables
- ✅ Vite como build tool

¡Estás listo para la Etapa 3: React Intermedio! 

Navegación

Anterior	Siguiente
← Week 05: Composición	→ Week 07: React Router

Rúbrica de Evaluación - Semana 06

Custom Hooks y Proyecto Integrador

Información General

Aspecto	Detalle
Semana	06 de 20
Etapas	2 - Fundamentos de React (Final)
Tema Principal	Custom Hooks y Proyecto Integrador
Duración	8 horas
Tipo Evaluación	Sumativa (proyecto 100%)

Objetivos Evaluables

1. Crear custom hooks reutilizables con TypeScript
2. Extraer lógica de componentes a hooks personalizados
3. Tipar hooks con generics para máxima flexibilidad
4. Integrar todos los conceptos de la Etapa 2 en un proyecto

Recursos de Aprendizaje (No Evaluados)

La teoría y los ejercicios son recursos de preparación para el proyecto. No son entregables evaluados, pero se recomienda completarlos para adquirir las competencias necesarias.

Teoría

- Custom hooks: fundamentos y anatomía
- Patrones de composición de hooks
- Ciclo de vida de componentes
- Configuración de Vite

Ejercicios Guiados

- Ejercicio 01: `useToggle` , `useCounter` - Hooks básicos
- Ejercicio 02: `useForm` , `useArray` - Estado complejo
- Ejercicio 03: `useFetch` , `useDebounce` - Efectos
- Ejercicio 04: Hooks genéricos con `<T>`
- Ejercicio 05: Composición de múltiples hooks

Proyecto (100%)

Instrumento: Dashboard Interactivo (Proyecto Etapa 2)

Funcionalidad (40%)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (0-5)
----------	----------------	-----------	----------------	--------------------

Custom hooks implementados	5+ hooks reutilizables	3-4 hooks	2 hooks	<2 hooks
Widgets interactivos	4+ widgets funcionales	3 widgets	2 widgets	<2 widgets
Context API (tema/usuario)	2+ contexts bien tipados	1 context completo	Context básico	Sin context
Manejo de estados	Loading/error/data	Estados básicos	Falta algún estado	No maneja estados

Adaptación al Dominio (35%)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (0-5)
Entidades del dominio	Todas coherentes	Mayoría coherente	Parcialmente coherente	Incoherente
Widgets adaptados	Específicos del dominio	Adaptados	Genéricos	Sin adaptar
Nomenclatura contextualizada	Nombres del dominio	Parcialmente	Pocos nombres	Genérica
Datos mock representativos	Realistas para el dominio	Aceptables	Básicos	Sin relación

Calidad del Código (25%)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (0-5)
TypeScript estricto	0 errores, tipos completos	Errores menores	any ocasional	Muchos any
Estructura de código	Separación clara, reutilizable	Organizado	Algo desorganizado	Desordenado
Documentación	JSDoc completo	Comentarios	Mínima	Sin documentar
Hooks bien encapsulados	Single responsibility	Aceptable	Hacen demasiado	No encapsulados

1234

Cálculo de Calificación

Calificación Final = (Funcionalidad × 0.40) + (Adaptación al Dominio × 0.35) + (Calidad del Código × 0.25)

Escala de Conversión

Puntos	Calificación	Nivel
95-100	10	Excepcional
85-94	9	Sobresaliente
75-84	8	Notable
70-74	7	Aprobado
60-69	6	Suficiente

< 60	5 o menos	No aprobado
------	-----------	-------------

Criterios de Aprobación

- **Mínimo 70%** en el proyecto para aprobar la semana
- Proyecto funcional y ejecutable
- Adaptación coherente al dominio asignado

✔ Checklist de Entrega

Preparación (No evaluada)

- ☐ Ejercicio 01: `useToggle` , `useCounter` implementados
- ☐ Ejercicio 02: `useForm` , `useArray` con estado complejo
- ☐ Ejercicio 03: `useFetch` , `useDebounce` con efectos
- ☐ Ejercicio 04: Hooks genéricos con `<T>`
- ☐ Ejercicio 05: Composición de múltiples hooks

📦 Proyecto Dashboard (100% - Evaluado)

- ☐ Repositorio con estructura organizada
- ☐ README con descripción del dominio asignado
- ☐ Mínimo 5 custom hooks reutilizables:
 - ☐ Hook de estado (toggle, counter, form)
 - ☐ Hook de efectos (fetch, debounce)
 - ☐ Hook de storage (localStorage)
 - ☐ Hook de UI (responsive, theme)
 - ☐ Hook compuesto (auth, dashboard data)
- ☐ Mínimo 3 widgets funcionales
- ☐ Context para tema (claro/oscurο)
- ☐ TypeScript sin errores (`tsc --noEmit`)
- ☐ Código documentado con comentarios JSDoc

🏛️ Adaptación por Dominio

El proyecto debe estar adaptado al dominio asignado:

Dominio	Widgets Sugeridos
📖 Biblioteca	Libros prestados, usuarios activos, búsqueda
💊 Farmacia	Stock bajo, ventas del día, alertas vencimiento
🏋️ Gimnasio	Miembros activos, clases hoy, ocupación
🏫 Escuela	Estudiantes, promedio notas, asistencia
🐾 Pet Shop	Productos populares, citas grooming, inventario
🍽️ Restaurante	Pedidos activos, mesas ocupadas, platos populares
🏦 Banco	Transacciones hoy, saldo total, alertas
🚕 Taxis	Viajes activos, conductores disponibles, ingresos

 Hospital	Pacientes hoy, camas disponibles, emergencias
 Cine	Películas hoy, butacas vendidas, próximos estrenos

Fechas Importantes

Actividad	Fecha Límite
Preparación (ejercicios)	Recomendado día 5
Proyecto Dashboard	Día 7 de la semana
Retroalimentación	Día 8-9

Nota: Solo el proyecto es entregable evaluado. Los ejercicios son preparación recomendada.

Retroalimentación

Fortalezas Comunes

- Hooks bien encapsulados y reutilizables
- Tipado correcto con generics
- Buena separación de concerns

Áreas de Mejora Frecuentes

- Olvidar cleanup en useEffect dentro de hooks
- No manejar todos los estados (loading, error)
- Hooks que hacen demasiadas cosas
- Dependencias incorrectas en useEffect

Recursos de Refuerzo

- [React Docs: Custom Hooks](#)
- [useHooks.com](#) - Colección de hooks
- [ahooks](#) - Librería de hooks

Competencias Etapa 2 Completadas

Al aprobar esta semana, el estudiante demuestra:

Competencia	Nivel Alcanzado
Componentes funcionales TypeScript	✅ Intermedio
Hooks nativos (useState, useEffect)	✅ Intermedio
Manejo de eventos y formularios	✅ Intermedio
Renderizado condicional y listas	✅ Intermedio
Context API	✅ Básico
Custom Hooks	✅ Básico
Configuración Vite	✅ Básico

Próximo paso: Etapa 3 - React Intermedio (React Router, Zustand, React Query)

